المذكرة الاحترافية

Software Engineering for Machine Learning Engineers

الفصل السادس: Error Handling

إزاي تتعامل مع الأخطاء بشكل احترافي بدل ما ينهار النظام بصمت أو بشكل غامض

Exceptions • Validation • Logging

جدول المحتويات

جدول المحتويات 2
مقدمة الفصل السادس: إيه اللي بيحصل لما حاجة تغلط؟ 4
٦،١ Exceptions — التعامل مع الاستثناءات 5
١. ليه التعامل الصحيح مع الاستثناءات مهم؟ 5
٢. المشكلة اللي بيحلها التعامل الصحيح مع Exceptions 5
٣. إزاي كانت الشركات بتتعامل قبل الاهتمام بمعالجة الاستثناءات 5
٤. ليه الطريقة القديمة مش مناسبة 5
٥. الفكرة الأساسية وراء التعامل الصحيح مع Exceptions 5
٦. المفاهيم الأساسية 5
٧. المكونات 5
٨. إزاي المبدأ ده بيشتغل داخليًا 5
٩. إزاي بيتم استخدامه داخل الشركات الكبرى 6
١٠. فين موقعه داخل مشروع ML حقيقي 6
١١. امتى تستخدم استثناءات مخصصة 6
١٢. امتى ممكن تكتفي بالاستثناءات العامة 6
١٣. أفضل الممارسات 6
١٤. أشهر الأخطاء الشائعة 6
١٥. اعتبارات الأداء 6
١٦. الاعتبارات الأمنية 6
١٧. اعتبارات قابلية التوسع 7
١٨. مقارنة مع نهج بديل 7
١٩. مثال عملي صغير 7
٢٠. مثال من مشروع Production حقيقي 7
٢١. ملخص الموضوع 7
٦،٢ Validation — التحقق من صحة البيانات 8
١. ليه Validation مهم؟ 8
٢. المشكلة اللي بيحلها Validation 8
٣. إزاي كانت الشركات بتتعامل قبل الاهتمام بـ Validation 8
٤. ليه الطريقة القديمة مش مناسبة 8
٥. الفكرة الأساسية وراء Validation 8
٦. المفاهيم الأساسية 8
٧. المكونات 8
٨. إزاي المبدأ ده بيشتغل داخليًا 8
٩. إزاي بيتم استخدامه داخل الشركات الكبرى 9
١٠. فين موقعه داخل مشروع ML حقيقي 9
١١. امتى تطبق Validation بصرامة 9
١٢. امتى ممكن تتساهل شوية 9
١٣. أفضل الممارسات 9
١٤. أشهر الأخطاء الشائعة 9
١٥. اعتبارات الأداء 9

١٦. الاعتبارات الأمنية 9
١٧. اعتبارات قابلية التوسع 9
١٨. مقارنة مع نهج بديل 10
١٩. مثال عملي صغير 10
٢٠. مثال من مشروع Production حقيقي 10
٢١. ملخص الموضوع 10
٦,٣ Logging — تسجيل الأحداث 11
١. ليه Logging مهم؟ 11
٢. المشكلة اللي بيحلها Logging 11
٣. إزاي كانت الشركات بتتعامل قبل الاهتمام بـ Logging المنظم 11
٤. ليه الطريقة القديمة مش مناسبة 11
٥. الفكرة الأساسية وراء Logging 11
٦. المفاهيم الأساسية 11
٧. المكونات 11
٨. إزاي المبدأ ده بيشتغل داخليًا 11
٩. إزاي بيتم استخدامه داخل الشركات الكبرى 12
١٠. فين موقعه داخل مشروع ML حقيقي 12
١١. امتى تستخدم مستوى Logging مفصل (Debug) 12
١٢. امتى تكتفي بمستوى Logging أعلى (Info أو Warning) 12
١٣. أفضل الممارسات 12
١٤. أشهر الأخطاء الشائعة 12
١٥. اعتبارات الأداء 12
١٦. الاعتبارات الأمنية 12
١٧. اعتبارات قابلية التوسع 13
١٨. مقارنة مع نهج بديل 13
١٩. مثال عملي صغير 13
٢٠. مثال من مشروع Production حقيقي 13
٢١. ملخص الموضوع 13
مشروع عملي على الفصل: بناء طبقة معالجة أخطاء متكاملة لـ Mini Training Pipeline 14
خطوات التنفيذ 14
تحديات شائعة عند التطبيق 14
إزاي الفصل ده بيتستخدم داخل مشروع Machine Learning حقيقي 14
ملخص الفصل السادس وربطه بالفصل القادم 14

مقدمة الفصل السادس: إيه اللي بيحصل لما حاجة تغلط؟

في الفصل اللي فات اتعلمنا إزاي ننظم المشروع فعليًا في مجلدات وملفات (Modular Design وPackages)، وإزاي ندير الإعدادات والقيم الحساسة (Configuration وEnvironment Variables). الفصل ده هيتعامل مع سؤال تاني بنفس الأهمية: إيه اللي بيحصل لما حاجة تغلط؟

مهما كان الكود منظم ونظيف، حاجات هتغلط لا محالة: ملف مش موجود، بيانات ناقصة أو بشكل غير متوقع، اتصال بالشبكة بيفشل، أو نموذج بياخد مدخلات مش صالحة. الفرق بين نظام احترافي ونظام هش مش إنه ميغلطش أبدًا (ده مستحيل)، لكن إزاي بيتعامل مع الغلط ده لما يحصل.

هندرس في الفصل ده تلات مواضيع مترابطة: Exceptions (إزاي تتعامل مع الأخطاء بشكل منظم بدل ما البرنامج ينهار فجأة)، Validation (إزاي تتأكد من صحة البيانات قبل ما تدخل النظام أصلًا)، وLogging (إزاي تسجل الأحداث المهمة عشان تقدر تفهم إيه اللي حصل لما مشكلة تظهر).

ليه بندرس الموضوع ده دلوقتي؟ لأن التعامل الصح مع الأخطاء هو الأساس اللي هنحتاجه في الفصل الجاي (Testing)، حيث هنتعلم إزاي نكتب اختبارات بتتأكد إن الكود بيتعامل صح مع الحالات الطبيعية والحالات الاستثنائية على حد سواء. من غير فهم واضح لإزاي نتعامل مع الأخطاء، صعب تكتب اختبار حقيقي وفعال.

٦,١ Exceptions — التعامل مع الاستثناءات

الاستثناء (Exception) هو آلية لغة البرمجة للإشارة إن حاجة غير متوقعة حصلت أثناء التنفيذ، والتعامل الاحترافي معاها بيحدد الفرق بين نظام بيفشل بشكل واضح ومفهوم، ونظام بينهار فجأة من غير أي تفسير.

١. ليه التعامل الصحيح مع الاستثناءات مهم؟

أي نظام حقيقي هيقابل حالات غير متوقعة: ملف مفقود، اتصال شبكة فشل، بيانات بشكل غلط. لو النظام مش مصمم للتعامل مع الحالات دي بشكل واضح، أي خطأ بسيط ممكن يوقف النظام كله فجأة من غير أي تفسير مفيد لسبب المشكلة.

في مشاريع ML، ده مهم جدًا لأن الـ Pipeline بيتكون من مراحل متعددة (تحميل بيانات، معالجة، تدريب، تقييم)، وفشل مرحلة واحدة لازم يتعامل معاه بشكل واضح من غير ما يخلي باقي المراحل تفشل بطرق غامضة وصعبة التشخيص.

٢. المشكلة اللي بيحلها التعامل الصحيح مع Exceptions

المشكلة هي البرامج اللي بتنهار بشكل مفاجئ برسالة خطأ تقنية غامضة (زي KeyError أو NoneType has no attribute) من غير أي سياق يوضح إيه اللي حصل فعلًا وإزاي تصلحه. المهندس بيضطر يقعد يتتبع الكود سطر سطر عشان يفهم مصدر المشكلة الحقيقي.

٣. إزاي كانت الشركات بتتعامل قبل الاهتمام بمعالجة الاستثناءات

كان شائع تجاهل احتمالية حدوث أخطاء تمامًا في الكود الأولي أو السكربتات السريعة، وكان الاعتماد بيبقى على إن البرنامج يفشل ويوقف تمامًا لما حاجة تغلط، من غير أي معالجة أو رسالة واضحة.

٤. ليه الطريقة القديمة مش مناسبة

الانهيار المفاجئ من غير معالجة بيخلي تشخيص المشكلة صعب جدًا، خصوصًا في أنظمة معقدة بيها عدة مراحل، وبيضيع وقت كبير في محاولة فهم السياق اللي أدى للخطأ ده، وقت كان ممكن يتوفر لو الخطأ اتعامل معاه واتوضح سببه من الأول.

٥. الفكرة الأساسية وراء التعامل الصحيح مع Exceptions

بدل ما نسيب الخطأ ينتشر بشكل غامض، بتمسكه في النقطة المناسبة (Try/Except)، وبتقرر إما تتعامل معاه بشكل مناسب (زي إعادة المحاولة أو استخدام قيمة افتراضية)، أو ترميه تاني كخطأ أوضح وأكتر تحديدًا يوضح السياق الحقيقي للمشكلة.

٦. المفاهيم الأساسية

- Custom Exceptions: تعريف أنواع استثناءات خاصة بالمشروع (زي InvalidTrainingDataError) بدل الاعتماد على الاستثناءات العامة بس.
- Exception Hierarchy: تنظيم الاستثناءات المخصصة في تسلسل هرمي منطقي، بحيث تقدر تمسك مجموعة استثناءات مرتبطة بسهولة.
- Fail Fast: مبدأ يقول إن الخطأ لازم يظهر بأسرع وقت ممكن وبأوضح رسالة ممكنة، بدل ما ينتشر بصمت ويظهر تأثيره في مكان بعيد تمامًا عن السبب الحقيقي.

٧. المكونات

- كتلة try/except لمسك الاستثناء في النقطة المناسبة.
- استثناءات مخصصة (Custom Exception Classes) بترث من Exception الأساسي وبتوضح نوع الخطأ بدقة.
- رسالة خطأ واضحة توضح إيه اللي حصل وإيه السياق المرتبط بيه.

٨. إزاي المبدأ ده بيشتغل داخليًا

لما يحصل خطأ، Python بيوقف التنفيذ العادي ويبدأ يدور على أقرب كتلة except تقدر تتعامل مع نوع الخطأ ده، بالمرور صعودًا عبر سلسلة استدعاء الدوال (Call Stack). لو مفيش كتلة تتعامل معاه، البرنامج بينهار برسالة الخطأ الافتراضية. مسك الاستثناء في النقطة المناسبة بيدي فرصة لمعالجته بشكل ذكي أو توضيحه بدل الانهيار الكامل.

٩. إزاي بيتم استخدامه داخل الشركات الكبرى

في منصات ML الكبرى، بيتم تعريف تسلسل هرمي كامل من الاستثناءات المخصصة (زي DataValidationError وModelTrainingError وDeploymentError)، بحيث أي جزء من النظام بيقدر يفرق بسهولة بين نوع المشكلة اللي حصلت ويتعامل معاها بالطريقة المناسبة، سواء بإعادة المحاولة أو التصعيد لفريق الدعم الفني.

١٠. فين موقعه داخل مشروع ML حقيقي

بيظهر في التعامل مع فشل تحميل البيانات (ملف مفقود أو تنسيق غلط)، فشل الاتصال بمصادر بيانات خارجية، فشل التقارب أثناء التدريب (Training Divergence)، وفشل الاتصال بخدمات النشر أو التخزين.

١١. امتى تستخدم استثناءات مخصصة

- لما محتاج تفرق بين أنواع مختلفة من الأخطاء بشكل واضح يسمح بالتعامل المختلف مع كل نوع.
- لما الاستثناء العام (زي ValueError) مش بيوضح كفاية سياق المشكلة الحقيقي.

١٢. امتى ممكن تكتفي بالاستثناءات العامة

- في سكربتات بسيطة جدًا مش متوقع لها تعامل معقد مع أنواع أخطاء مختلفة.
- لما الاستثناء العام الموجود بالفعل (زي ValueError) بيوضح المشكلة بشكل كافٍ من غير الحاجة لنوع مخصص جديد.

١٣. أفضل الممارسات

Best Practices

- امسك الاستثناء في أضيق نطاق ممكن (Try Block محدد)، متمسكش استثناءات عامة على كتلة كود كبيرة جدًا.
- استخدم استثناءات مخصصة توضح نوع المشكلة بدقة بدل الاعتماد على استثناءات عامة طول الوقت.
- اكتب رسالة خطأ واضحة تتضمن السياق المفيد (زي اسم الملف أو القيمة اللي سببت المشكلة).

١٤. أشهر الأخطاء الشائعة

Common Mistakes

- استخدام except: عام يمسك كل أنواع الأخطاء من غير تمييز، وده بيخفي أخطاء حقيقية كان المفروض تظهر وتتصلح.
- تجاهل الاستثناء تمامًا (Silent Failure) عن طريق كتلة except فاضية من غير أي تسجيل أو معالجة.
- رسائل خطأ غامضة ما بتوضحش السياق الفعلي للمشكلة، زي An error occurred من غير أي تفاصيل إضافية.

١٥. اعتبارات الأداء

معالجة الاستثناءات في Python بتكلفة أداء مهملة في الحالة العادية (لما مفيش استثناء بيحصل فعلًا)، لكن رفع ومعالجة الاستثناء نفسه لما يحصل فعلًا له تكلفة أعلى نسبيًا، فمن الأفضل عدم استخدام الاستثناءات للتحكم في التدفق العادي للبرنامج (Control Flow) في الحالات المتوقعة والمتكررة بشكل كبير.

١٦. الاعتبارات الأمنية

لازم الانتباه لعدم كشف تفاصيل حساسة في رسائل الخطأ اللي بتظهر للمستخدم النهائي (زي تفاصيل داخلية عن بنية قاعدة البيانات)، بينما التفاصيل الكاملة ممكن تتسجل بأمان في نظام الـ Logging الداخلي بس.

١٧. اعتبارات قابلية التوسع

تسلسل هرمي واضح من الاستثناءات المخصصة بيسمح بإضافة أنواع أخطاء جديدة بسهولة مع نمو النظام، من غير التأثير على المعالجة الموجودة بالفعل للأنواع التانية من الأخطاء.

١٨. مقارنة مع نهج بديل

البديل هو إرجاع قيم خاصة (زي None أو -١) للإشارة لحدوث خطأ بدل رفع استثناء. الأسلوب ده بيتطلب فحص القيمة المرجعة في كل مرة، وسهل تنسى تفحصها، بينما الاستثناء بيجبر التعامل معاه (أو الانهيار الواضح لو محدش تعامل معاه)، وده بيخليه أوضح وأقل عرضة للتجاهل بالخطأ.

١٩. مثال عملي صغير

```
class DataValidationError(Exception):
    """Raised when training data fails validation checks."""
    pass

def load_training_data(path: str):
    try:
        data = read_csv(path)
    except FileNotFoundError as e:
        raise DataValidationError(
            f'Training data file not found at: {path}'
        ) from e

    if data.empty:
        raise DataValidationError(
            f'Training data file is empty: {path}'
        )
    return data

try:
    data = load_training_data('data/training.csv')
except DataValidationError as e:
    logger.error(f'Cannot proceed with training: {e}')
    raise
```

٢٠. مثال من مشروع Production حقيقي

في نظام تدريب حقيقي، لما التدريب يفشل بسبب تقارب غير مستقر (Training Divergence)، النظام بيرمي استثناء مخصص TrainingDivergenceError بدل ما ينهار بشكل غامض، وده بيسمح للنظام الأكبر يتعامل مع الحالة دي بذكاء، زي إعادة المحاولة بـ Learning Rate أقل تلقائيًا.

٢١. ملخص الموضوع

خلاصة Exceptions

- امسك الاستثناءات في أضيق نطاق ممكن، وتجنب except العام الشامل.
- استخدم استثناءات مخصصة توضح نوع المشكلة بدقة.
- اكتب رسائل خطأ واضحة تتضمن السياق المفيد لتسهيل التشخيص.

٦,٢ Validation — التحقق من صحة البيانات

Validation هي عملية التأكد من إن البيانات الداخلة للنظام صالحة ومطابقة للتوقعات، قبل ما تتفاعل مع باقي أجزاء النظام. الفكرة هنا إنك تمسك المشكلة أقرب ما يمكن لمصدرها، بدل ما تسيبها تنتشر وتظهر آثارها في مكان بعيد وغامض.

١. ليه Validation مهم؟

لو بيانات غير صالحة (زي قيمة سالبة في عمود المفروض يكون موجب، أو نوع بيانات غلط) دخلت النظام من غير تحقق، المشكلة ممكن متظهرش فورًا، وإنما تظهر بعد كذا مرحلة في الـ Pipeline بشكل غامض وصعب ربطه بمصدره الحقيقي.

في مشاريع ML، Validation مهم بشكل خاص لأن جودة البيانات بتؤثر مباشرة على جودة النموذج، والمشكلة اللي بتنتج عن بيانات سيئة ممكن تظهر بس بعد ساعات من التدريب، وقت كان ممكن يتوفر لو التحقق حصل من البداية.

٢. المشكلة اللي بيحلها Validation

المشكلة هي البيانات الفاسدة أو غير المتوقعة اللي بتدخل النظام من غير فحص، وبتسبب أخطاء غامضة في مراحل لاحقة بعيدة تمامًا عن مصدر المشكلة الحقيقي، زي نموذج بيدرب على بيانات فيها قيم مفقودة من غير معالجة، وينتج نتائج غير منطقية من غير أي تفسير واضح ليه.

٣. إزاي كانت الشركات بتتعامل قبل الاهتمام بـ Validation

كان شائع افتراض إن البيانات الداخلة دايمًا نظيفة وصحيحة الشكل، خصوصًا في السكربتات السريعة والتجارب الأولية، من غير أي تحقق حقيقي من صحتها قبل استخدامها.

٤. ليه الطريقة القديمة مش مناسبة

الافتراض إن البيانات دايمًا نظيفة بيخالف الواقع في أي نظام حقيقي متصل بمصادر بيانات خارجية، وأي بيانات غير متوقعة بتدخل النظام بتسبب مشاكل صعبة التشخيص لاحقًا، بدل ما تتوقف من البداية برسالة واضحة.

٥. الفكرة الأساسية وراء Validation

قبل ما البيانات تدخل أي منطق معالجة أساسي، بتتحقق من مطابقتها لمجموعة قواعد محددة (النوع الصحيح، النطاق المسموح، القيم المطلوبة موجودة). لو البيانات مش مطابقة، بترفض فورًا برسالة واضحة، بدل ما تكمل وتسبب مشاكل لاحقًا.

٦. المفاهيم الأساسية

- Schema Validation: التحقق من إن البيانات مطابقة لهيكل محدد مسبقًا (أنواع الأعمدة، الحقول المطلوبة).
- Business Rule Validation: التحقق من قواعد منطقية خاصة بمجال العمل (زي عمر المستخدم لازم يكون رقم موجب أقل من ١٥٠).
- Fail Fast Validation: رفض البيانات غير الصالحة في أقرب نقطة ممكنة، بدل السماح لها بالمرور والتسبب في مشاكل لاحقة.

٧. المكونات

- تعريف واضح للقواعد المطلوب التحقق منها (النوع، النطاق، الحقول المطلوبة).
- دالة أو كلاس Validator مسؤول عن تنفيذ الفحوصات دي.
- استثناء واضح (زي DataValidationError) بيترفع لو البيانات مش مطابقة، مع رسالة توضح بالظبط إيه اللي غلط.

٨. إزاي المبدأ ده بيشتغل داخليًا

قبل ما البيانات تدخل منطق المعالجة الأساسي، بتمر على طبقة Validation اللي بتفحص كل قاعدة من القواعد المطلوبة. لو أي قاعدة اتخالفت، العملية بتتوقف فورًا برسالة واضحة، وده بيمنع البيانات الفاسدة من الوصول لأي منطق تاني في النظام.

٩. إزاي بيتم استخدامه داخل الشركات الكبرى

في منصات ML الكبرى، بيتم استخدام أدوات متخصصة للتحقق من صحة البيانات (زي Great Expectations أو Pandera) بتفحص كل دفعة بيانات جديدة قبل دخولها الـ Pipeline، وبتولد تقرير واضح يوضح أي انتهاكات للقواعد المتوقعة قبل ما تؤثر على عملية التدريب.

١٠. فين موقعه داخل مشروع ML حقيقي

بيظهر في التحقق من صحة بيانات التدريب قبل استخدامها، التحقق من صحة المدخلات الواردة لنموذج منشور في الإنتاج (Serving Time Validation)، والتحقق من صحة الإعدادات (Configuration) نفسها قبل بدء أي عملية تدريب.

١١. امتى تطبق Validation بصرامة

- في أي نقطة دخول للبيانات من مصدر خارجي (ملف، API، قاعدة بيانات).
- في أي نظام Production متصل مباشرة بمستخدمين أو أنظمة خارجية غير موثوقة تمامًا.

١٢. امتى ممكن تتساهل شوية

- في تحليل استكشافي سريع لبيانات موثوقة ومعروفة المصدر بالفعل، فيه فحص Validation الكامل ممكن يكون زيادة عن اللزوم للسرعة.

١٣. أفضل الممارسات

Best Practices

- طبق مبدأ Fail Fast: افحص البيانات في أقرب نقطة دخول ممكنة، مش بعد ما تمر على عدة مراحل معالجة.
- اكتب رسائل خطأ توضح بالظبط أي قاعدة اتخالفت وليه.
- استخدم أدوات متخصصة للتحقق من صحة البيانات (زي Pandera) بدل كتابة فحوصات يدوية متفرقة.

١٤. أشهر الأخطاء الشائعة

Common Mistakes

- افتراض إن البيانات دايمًا صحيحة الشكل من غير أي فحص حقيقي.
- فحص جزء بسيط بس من القواعد المطلوبة (زي فحص النوع بس من غير فحص النطاق المنطقي للقيمة).
- تأخير الفحص لمرحلة متأخرة في الـ Pipeline بدل أقرب نقطة دخول ممكنة.

١٥. اعتبارات الأداء

فحص Validation بيضيف تكلفة حسابية إضافية، خصوصًا مع كميات بيانات ضخمة. في الحالات دي، ممكن تحتاج توازن بين شمولية الفحص وسرعته، زي فحص عينة من البيانات بدل كل صف على حدة في حالات معينة، حسب حجم البيانات والمخاطرة المقبولة.

١٦. الاعتبارات الأمنية

Validation جزء أساسي من الأمان: التحقق من صحة أي مدخلات جاية من مصدر خارجي (خصوصًا من المستخدمين) بيمنع هجمات زي حقن الأكواد (Injection Attacks) أو استغلال حالات حدية غير متوقعة في النظام.

١٧. اعتبارات قابلية التوسع

طبقة Validation منظمة وواضحة بتسمح بإضافة قواعد تحقق جديدة بسهولة مع تطور متطلبات جودة البيانات، من غير التأثير على باقي منطق معالجة البيانات.

١٨. مقارنة مع نهج بديل

البديل هو الاعتماد على معالجة الأخطاء لاحقًا (Reactive Error Handling) بدل التحقق المسبق، يعني تسيب البيانات الفاسدة تدخل وتتعامل مع المشكلة لما تظهر. الأسلوب ده بيؤدي لتشخيص أصعب بكتير للمشاكل، والشركات الكبرى بتفضل التحقق الاستباقي (Proactive Validation) لأنه بيمنع المشكلة من الأساس بدل التعامل مع نتائجها لاحقًا.

١٩. مثال عملي صغير

```
class DataValidationError(Exception):
    pass

def validate_training_data(df):
    required_columns = {'age', 'income', 'label'}
    missing = required_columns - set(df.columns)
    if missing:
        raise DataValidationError(f'Missing required columns: {missing}')

    if (df['age'] < 0).any() or (df['age'] > 150).any():
        raise DataValidationError('Column age contains values outside valid range [0,
150]')

    if df['income'].isnull().any():
        raise DataValidationError('Column income contains missing values')

    return True

df = load_training_data('data/training.csv')
validate_training_data(df)
# Only proceed to training if validation passes
```

٢٠. مثال من مشروع Production حقيقي

في نظام ML بيستقبل بيانات من عدة مصادر خارجية، طبقة Validation بتفحص كل دفعة بيانات جديدة قبل دخولها لعملية التدريب، وبتولد تنبيه فوري للفريق المسؤول لو نسبة القيم المفقودة تجاوزت حد معين، بدل ما يكتشف الفريق المشكلة بعد أيام من ملاحظة تدهور غير مفسر في أداء النموذج.

٢١. ملخص الموضوع

خلاصة Validation

- افحص البيانات في أقرب نقطة دخول ممكنة، وارفض الغير صالح فورًا برسالة واضحة.
- غطِ كل القواعد المهمة: النوع، النطاق، الحقول المطلوبة، مش جزء بسيط بس منها.
- Validation جزء أساسي من الأمان، مش بس من جودة البيانات.

٦,٣ Logging — تسجيل الأحداث

Logging هو عملية تسجيل الأحداث المهمة اللي بتحصل أثناء تشغيل النظام، بحيث تقدر تفهم إيه اللي حصل بالظبط لما مشكلة تظهر، من غير الحاجة لإعادة إنتاج المشكلة يدويًا أو التخمين.

١. ليه Logging مهم؟

لما نظام يشتغل في Production، مفيش حد بيراقب كل سطر كود بيتنفذ لحظة بلحظة. لو مشكلة حصلت، الطريقة الوحيدة لفهم إيه اللي حصل هي الرجوع للسجلات (Logs) اللي اتسجلت وقت التنفيذ. من غير Logging كافٍ، تشخيص أي مشكلة في الإنتاج بيصبح شبه مستحيل.

في مشاريع ML، Logging مهم بشكل خاص لتتبع تقدم التدريب (زي قيمة الخسارة بعد كل Epoch)، ولتشخيص سبب فشل أي مرحلة في الـ Pipeline بعد ما تكون خلصت وانتقلت لمرحلة تانية.

٢. المشكلة اللي بيحلها Logging

المشكلة هي الأنظمة اللي بتشتغل بصمت من غير أي تسجيل للأحداث المهمة، وأي مشكلة بتحصل بيصبح صعب معرفة سببها بعد ما تكون حصلت، لأن مفيش أي أثر مسجل يوضح إيه اللي كان بيحصل في النظام لحظة المشكلة.

٣. إزاي كانت الشركات بتتعامل قبل الاهتمام بـ Logging المنظم

كان شائع الاعتماد على أوامر print() بسيطة متناثرة في الكود للتصحيح السريع أثناء التطوير، وبتتشال قبل النشر للإنتاج، وده بيسيب النظام من غير أي أثر مفيد وقت حدوث مشكلة حقيقية بعيدًا عن بيئة التطوير.

٤. ليه الطريقة القديمة مش مناسبة

أوامر print() المتناثرة مش منظمة، ومافيهاش مستويات أهمية (زي التفرقة بين معلومة عادية وتحذير وخطأ خطير)، ومش سهل البحث فيها أو تحليلها بشكل منهجي، وغالبًا بتتشال قبل النشر للإنتاج بالظبط في الوقت اللي هتكون أكتر فايدة فيه.

٥. الفكرة الأساسية وراء Logging

بتستخدم نظام Logging منظم بيسجل الأحداث المهمة مع مستوى أهمية واضح (Debug، Info، Warning، Error، Critical)، ومعلومات سياقية إضافية (الوقت، مصدر الحدث)، بحيث السجلات دي تبقى مفيدة ومنظمة وقابلة للبحث والتحليل لاحقًا.

٦. المفاهيم الأساسية

- Log Levels: مستويات أهمية مختلفة (Debug للتفاصيل الدقيقة، Info للأحداث العادية، Warning للحالات المشبوهة، Error للأخطاء الفعلية، Critical للمشاكل الخطيرة جدًا).
- Structured Logging: تسجيل الأحداث بتنسيق منظم (زي JSON) بدل نص حر، وده بيسهل البحث والتحليل الآلي للسجلات لاحقًا.
- Contextual Information: إضافة معلومات سياقية مفيدة لكل سجل (زي معرف التجربة أو معرف المستخدم) تسهل ربط الأحداث ببعض.

٧. المكونات

- كائن Logger مُعرف لكل وحدة أو جزء من النظام.
- مستوى Logging محدد لكل بيئة (تفاصيل كتيرة في التطوير، تفاصيل أقل وأكثر أهمية في الإنتاج).
- وجهة تسجيل (Destination) زي ملف، أو نظام تجميع سجلات مركزي في بيئات الإنتاج الكبيرة.

٨. إزاي المبدأ ده بيشتغل داخليًا

كل جزء من النظام بيستخدم كائن Logger مخصص له، وبيسجل الأحداث المهمة بمستوى الأهمية المناسب. نظام Logging بيقرر بناءً على الإعدادات الحالية (زي بيئة التطوير أو الإنتاج) أي المستويات فعلًا هتتسجل وأي وجهة هتتسجل ليها، من غير الحاجة لتغيير الكود نفسه.

٩. إزاي بيتم استخدامه داخل الشركات الكبرى

في الشركات الكبرى، السجلات من كل الخدمات والأنظمة بتتجمع في نظام مركزي (زي Elasticsearch أو Splunk)، بحيث المهندسين يقدروا يبحثوا ويحللوا السجلات من كل النظام في مكان واحد، ويربطوا الأحداث المرتبطة ببعض عبر خدمات مختلفة عن طريق معرفات تتبع مشتركة (Trace IDs).

١٠. فين موقعه داخل مشروع ML حقيقي

بيظهر في تسجيل تقدم التدريب (قيمة الخسارة والمقاييس بعد كل Epoch)، تسجيل بداية ونهاية كل مرحلة في الـ Pipeline، تسجيل أي تحذيرات حول جودة البيانات، وتسجيل أي أخطاء حصلت مع السياق الكامل المرتبط بيها.

١١. امتى تستخدم مستوى Logging مفصل (Debug)

- في بيئة التطوير، لتتبع تفاصيل دقيقة أثناء تصحيح مشكلة معينة.
- مؤقتًا في بيئة الإنتاج، لما بتحقق في مشكلة محددة ومحتاج تفاصيل إضافية بشكل مؤقت.

١٢. امتى تكتفي بمستوى Logging أعلى (Info أو Warning)

- في بيئة الإنتاج بشكل دائم، لتجنب إغراق السجلات بتفاصيل كتيرة غير ضرورية بتؤثر على الأداء وتصعب البحث.

١٣. أفضل الممارسات

Best Practices

- استخدم مستويات Logging المناسبة لكل نوع حدث، ومتسجلش كل حاجة بمستوى Info بشكل عشوائي.
- أضف معلومات سياقية مفيدة لكل سجل (زي معرف التجربة) تسهل ربط الأحداث ببعض لاحقًا.
- استخدم Structured Logging (زي JSON) في بيئات الإنتاج لتسهيل البحث والتحليل الآلي.

١٤. أشهر الأخطاء الشائعة

Common Mistakes

- استخدام print() بدل نظام Logging منظم، خصوصًا في كود متوقع يدخل الإنتاج.
- تسجيل بيانات حساسة (زي كلمات مرور أو معلومات شخصية) في السجلات من غير حماية.
- استخدام نفس مستوى الأهمية (زي Info) لكل شيء، بدون تفرقة بين حدث عادي وتحذير وخطأ فعلي.

١٥. اعتبارات الأداء

تسجيل كمية كبيرة جدًا من السجلات، خصوصًا بمستوى Debug في بيئة إنتاج مزدحمة، ممكن يؤثر على الأداء ويستهلك مساحة تخزين كبيرة. لازم توازن بين شمولية التسجيل وتأثيره على الأداء، عن طريق ضبط مستوى Logging المناسب لكل بيئة.

١٦. الاعتبارات الأمنية

لازم الانتباه الشديد لعدم تسجيل بيانات حساسة (كلمات مرور، مفاتيح API، بيانات شخصية للمستخدمين) في السجلات، لأن السجلات غالبًا بتتحفظ لفترة طويلة ويقدر يوصلها عدد أكبر من الأشخاص مقارنة بالنظام نفسه.

١٧. اعتبارات قابلية التوسع

نظام Logging منظم ومركزي بيسمح بالتوسع لمراقبة عدد كبير من الخدمات والأنظمة المختلفة في مكان واحد، وده أساسي في الأنظمة الكبيرة اللي بتتكون من عشرات أو مئات الخدمات المنفصلة.

١٨. مقارنة مع نهج بديل

البديل هو الاعتماد على أوامر print() المتناثرة أو عدم التسجيل خالص. الأسلوب ده أسرع للتصحيح السريع أثناء التطوير المحلي، لكنه بيفشل تمامًا في بيئة الإنتاج حيث محدش بيراقب الشاشة مباشرة. الشركات الكبرى بتستثمر في بنية تحتية كاملة لتجميع وتحليل السجلات المركزية.

١٩. مثال عملي صغير

```
import logging

logger = logging.getLogger(__name__)
logging.basicConfig(level=logging.INFO)

def train_model(config):
    logger.info(f'Starting training with config: {config}')
    try:
        for epoch in range(config['epochs']):
            loss = run_epoch()
            logger.info(f'Epoch {epoch}: loss={loss:.4f}')
            if loss > 1e6:
                logger.warning(f'Loss diverging at epoch {epoch}')
    except Exception as e:
        logger.error(f'Training failed: {e}', exc_info=True)
        raise
    logger.info('Training completed successfully')
```

٢٠. مثال من مشروع Production حقيقي

في نظام ML بيشتغل على منصة سحابية، كل تجربة تدريب بيكون ليها معرف فريد (Experiment ID) بيتضاف كسياق لكل سجل مرتبط بيها، بحيث لو حصلت مشكلة، الفريق يقدر يبحث بسهولة عن كل الأحداث المرتبطة بالتجربة دي تحديدًا عبر نظام تجميع السجلات المركزي، من غير الحاجة لفلترة يدوية معقدة.

٢١. ملخص الموضوع

خلاصة Logging

- استخدم نظام Logging منظم بدل print()، مع مستويات أهمية واضحة لكل نوع حدث.
- أضف سياق مفيد (زي معرف التجربة) يسهل ربط الأحداث ببعض لاحقًا.
- انتبه لعدم تسجيل بيانات حساسة، وضبط مستوى التسجيل المناسب لكل بيئة.

مشروع عملي على الفصل: بناء طبقة معالجة أخطاء متكاملة لـ Mini Training Pipeline

المشروع ده هدفه إنك تدمج الثلاث مواضيع (Exceptions، Validation، Logging) في تصميم واحد متكامل لحماية Pipeline تدريب بسيط من الأخطاء الشائعة.

السيناريو: عندك Pipeline بيحمل بيانات من ملف، بيدربها، وبيحفظ النتيجة. عايز تضمن إن أي مشكلة (ملف مفقود، بيانات فاسدة، فشل أثناء التدريب) تتعامل معاها بشكل واضح ومسجل بدل ما تنهار النظام بشكل غامض.

خطوات التنفيذ

- عرف تسلسل هرمي من الاستثناءات المخصصة: PipelineError كأساس، وDataValidationError وTrainingError كفروع منه.
- طبق Validation على البيانات المحملة فور تحميلها، قبل ما تدخل أي منطق معالجة أو تدريب.
- أضف Logging في كل مرحلة رئيسية من الـ Pipeline: بداية ونهاية كل مرحلة، وأي تحذيرات أو أخطاء بمستوى الأهمية المناسب.
- تأكد إن كل رسالة خطأ بتتضمن سياق كافٍ (اسم الملف، القيمة المشكلة) يسهل التشخيص السريع.
- اختبر الـ Pipeline بسيناريوهات فشل مختلفة (ملف مفقود، عمود ناقص، قيمة خارج النطاق) وتأكد إن كل سيناريو بيتعامل معاه بوضوح.

تحديات شائعة عند التطبيق

- الإفراط في مسك الاستثناءات: مسك كل استثناء ممكن في كل مكان بيخفي أخطاء حقيقية كان المفروض تظهر وتتصلح فورًا. امسك بس الاستثناءات اللي فعلًا تقدر تتعامل معاها بشكل مفيد.
- الخلط بين Validation ومعالجة الاستثناءات: Validation بيمنع البيانات الفاسدة من الدخول من الأساس، بينما معالجة الاستثناءات بتتعامل مع أخطاء غير متوقعة أثناء التنفيذ. الاتنين مكملين لبعض، مش بديل عن بعض.
- الإسراف في التسجيل (Log Noise): تسجيل كل تفصيلة صغيرة بمستوى Info بيغرق السجلات المفيدة وسط ضجيج كبير، وبيصعب لقاء المعلومة المهمة وقت الحاجة الفعلية ليها.

إزاي الفصل ده بيتستخدم داخل مشروع Machine Learning حقيقي

التعامل الاحترافي مع الأخطاء بيظهر في كل جزء من نظام ML حقيقي: Exceptions مخصصة بتوضح فشل مرحلة معينة في الـ Pipeline، Validation بتضمن جودة البيانات قبل التدريب، وLogging بيوفر أثر كامل يوضح إيه اللي حصل في أي تجربة تدريب.

في الشركات الكبرى، أنظمة ML بتُعتبر جاهزة للإنتاج (Production-Ready) بس لما يكون فيها معالجة أخطاء شاملة زي دي، لأن مشاكل البيانات والبنية التحتية أمر لا مفر منه في أي نظام حقيقي يتفاعل مع مصادر بيانات وخدمات خارجية باستمرار.

ملخص الفصل السادس وربطه بالفصل القادم

في الفصل ده اتعلمت إزاي تتعامل مع الأخطاء بشكل احترافي: Exceptions لمسك ومعالجة الأخطاء بشكل منظم بدل الانهيار الغامض، Validation لمنع البيانات الفاسدة من الدخول للنظام من الأساس، وLogging لتوثيق الأحداث المهمة بحيث تقدر تفهم إيه اللي حصل وقت حدوث مشكلة.

الفصل الجاي (Testing) هيبني مباشرة على المفاهيم دي. هنتعلم إزاي نكتب اختبارات (Unit Testing وIntegration Testing) بتتأكد إن الكود بيتعامل صح مع الحالات الطبيعية، وكمان مع حالات الفشل والاستثناءات اللي اتعلمناها هنا. من غير فهم واضح لأنواع الأخطاء المحتملة وإزاي المفروض نتعامل معاها، صعب تكتب اختبار حقيقي يغطي كل السيناريوهات المهمة.